

Learning Objectives

By the end of this reading, you will be able to:

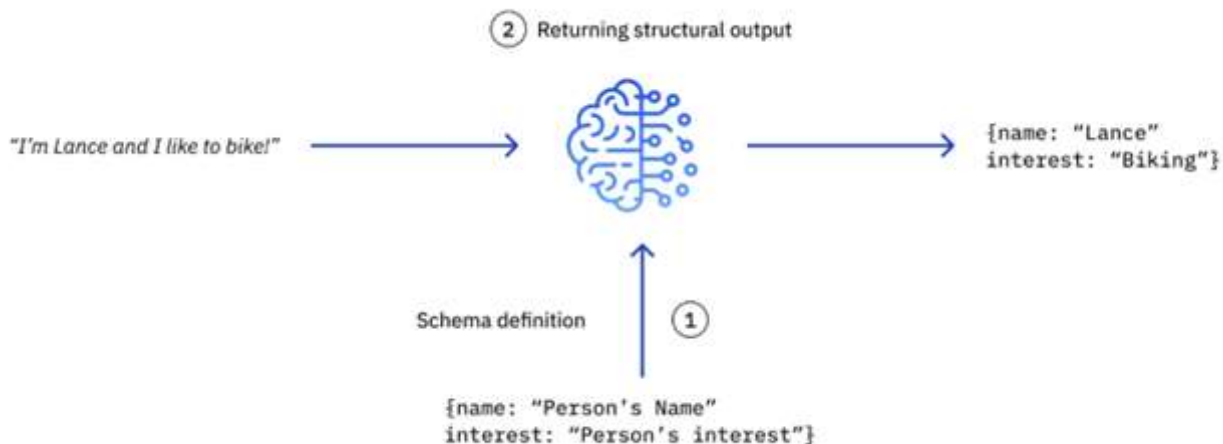
- Define structured output schemas using JSON-like formats and Pydantic models to enforce data consistency in LLM-generated responses.
- Implement tool calling and JSON mode to generate and validate structured outputs for use in databases, APIs, and application workflows.

This reading explores the use of structured outputs in large language model (LLM) applications, crucial for producing consistent, schema-compliant responses. Learners will discover how to implement output schemas using Pydantic models, tool calling, and JSON mode to support reliable database integration, API workflows, and data validation across various applications.

While many LLM applications respond to users in natural language, there are scenarios where we need models to output in a structured format. Structured outputs allow you to:

- Store model responses in databases with consistent formats
- Process outputs programmatically in your application
- Ensure responses include specific fields in a predictable structure
- Validate output against a defined schema

The process involves two main steps, as illustrated in the image:



1. **Schema Definition:** Defining the structure that we want the output to have.
2. **Returning structured output:** Having the model generate output conforming to the schema.

For example, given an input like *"I'm Lance and I like to bike,"* we can define a schema with fields for *"name"* and *"interest"*. The LLM processes this input against the schema and returns a structured output: `{name: "Lance", interest: "Biking"}`.

Schema definition

The schema defines the structure your model's output should follow. In LangChain, you have several options for defining schemas:

1. **JSON like structures:** The simplest format is a JSON-like structure, represented in Python as a dictionary or list.
2. **Pydantic models:** Pydantic is particularly useful for defining structured output schemas because it offers type hints and validation. Pydantic models offer several advantages as well, such as:
 - Type validation
 - Clear field descriptions
 - Built-in documentation
 - Integration with LangChain's tooling

Returning structured output

There are two main approaches to ensure models return structured outputs:

1. Tool calling

- Creating a model
- Binding the ResponseFormatter schema as a tool to the model
- Invoking the model with a user query
- Extracting the tool call arguments as a dictionary
- Optionally, parsing the dictionary into a pydantic object

2. JSON mode

- Requires using a model that supports JSON mode
- Uses the `with_structured_output` method with the `"json_mode"` parameter
- Returns results already formatted as a Python dictionary

Structured Output Method: `with_structured_output()`

